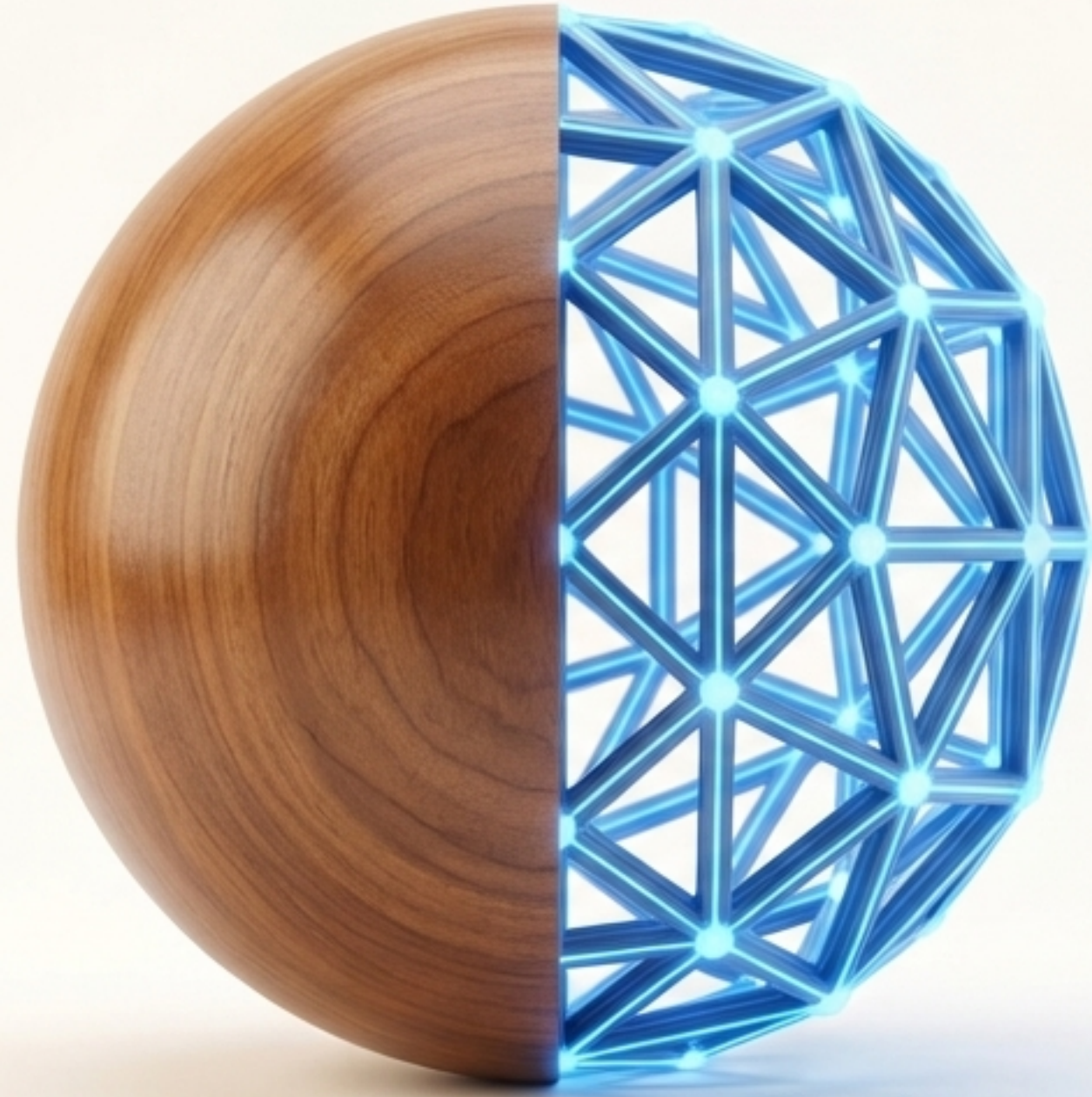




The Two Faces of JavaScript Data

Understanding Passing by Reference vs. Passing by Value

A Seemingly Simple Action with a Surprising Result

We create an array `arr`, and create a “copy” called `new_arr`. When we change an element in `new_arr`, something unexpected happens to the original `arr`. Why?

```
let arr = [1, 2, 3];  
let new_arr = arr;
```

```
new_arr[1] = 500;  
arr[0] = 400;
```

```
console.log(arr);  
console.log(new_arr);
```

```
> [400, 500, 3]  
> [400, 500, 3]
```


The Scene of the Crime: A Look Inside JavaScript's Memory



The Stack

A space for storing static data. Think of it as a list of labels or signposts. When a variable is created, it gets a spot on the stack.



The Heap

A larger, more dynamic space for storing complex data like objects and arrays. The actual data lives here.

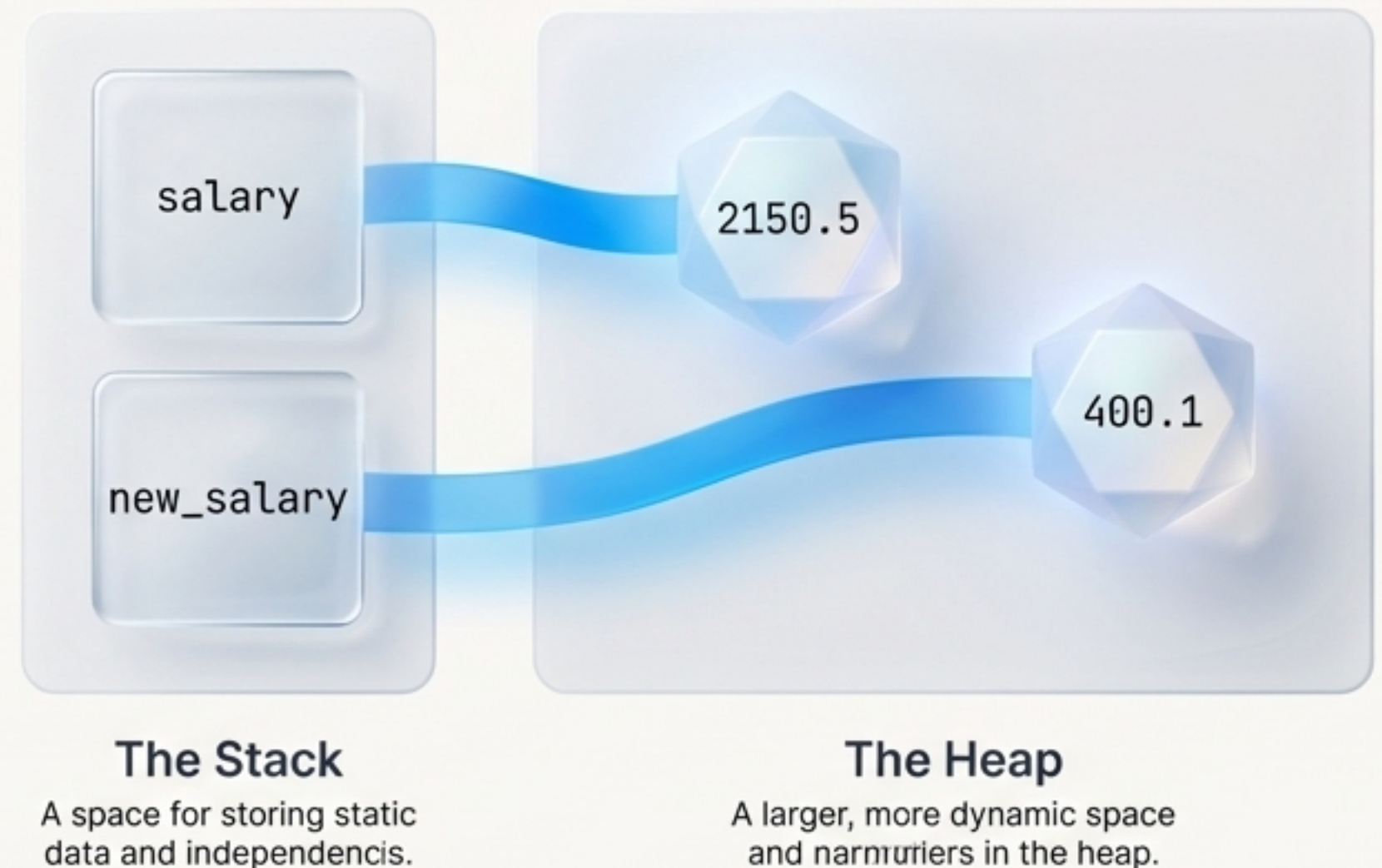


The Stack holds the variable *name*, but the Heap holds the actual complex *data*. An arrow connects them.

The Straightforward Citizen: Primitives are Passed by Value

Primitives like numbers, booleans, and strings are simple. When you copy them, you get a completely new, independent value. Changing one does not affect the other.

```
let salary = 2150.5;  
let new_salary = salary;  
new_salary = 400.1;  
  
console.log(salary);    // > 2150.5  
console.log(new_salary); // > 400.1
```

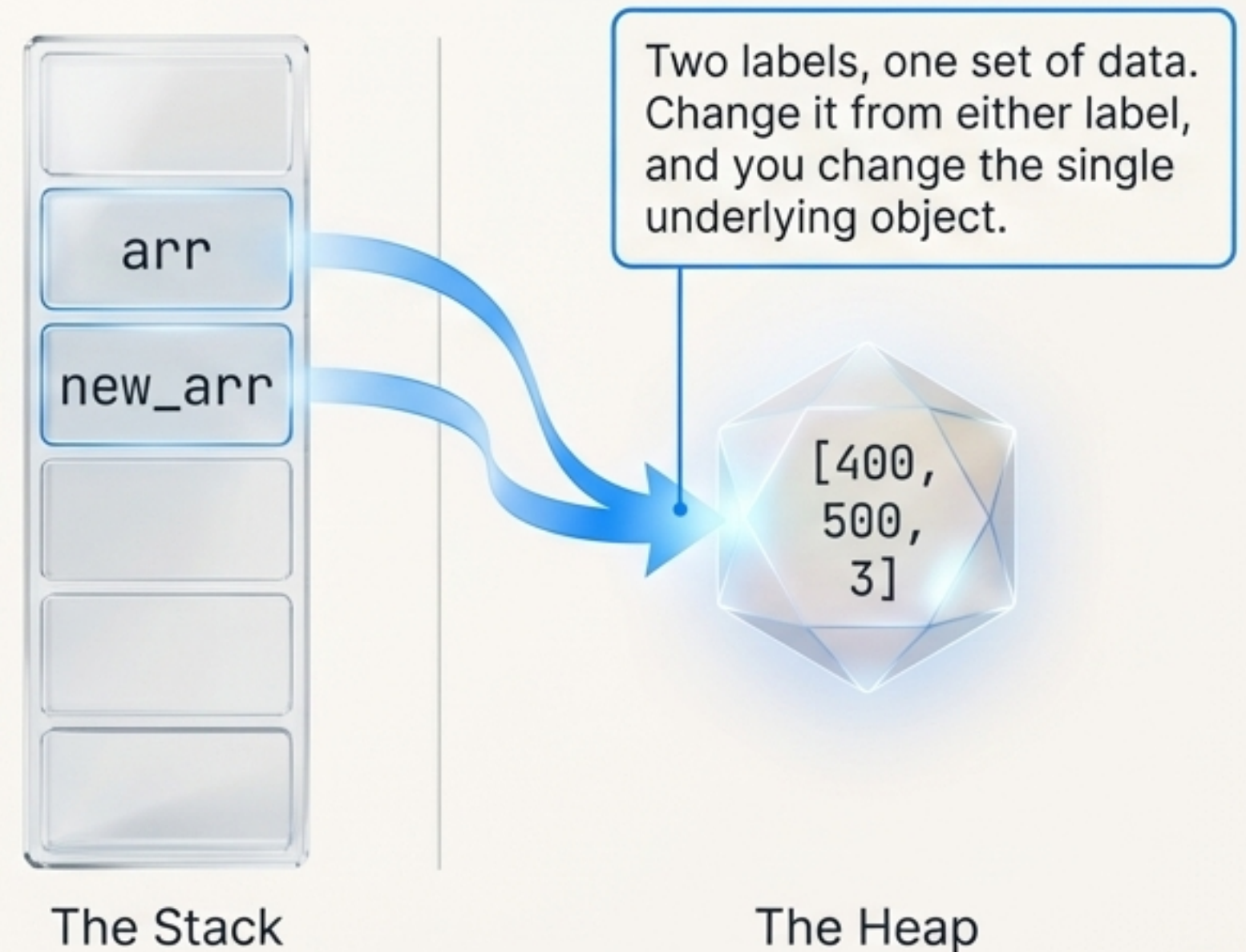


The Twist: Objects are Passed by Reference

This is the solution to our mystery. When you “copy” an object or array with `=`, you are not copying the data. You are only copying the address—the pointer to the data’s location on the Heap.

```
let arr = [1, 2, 3];  
let new_arr = arr;
```

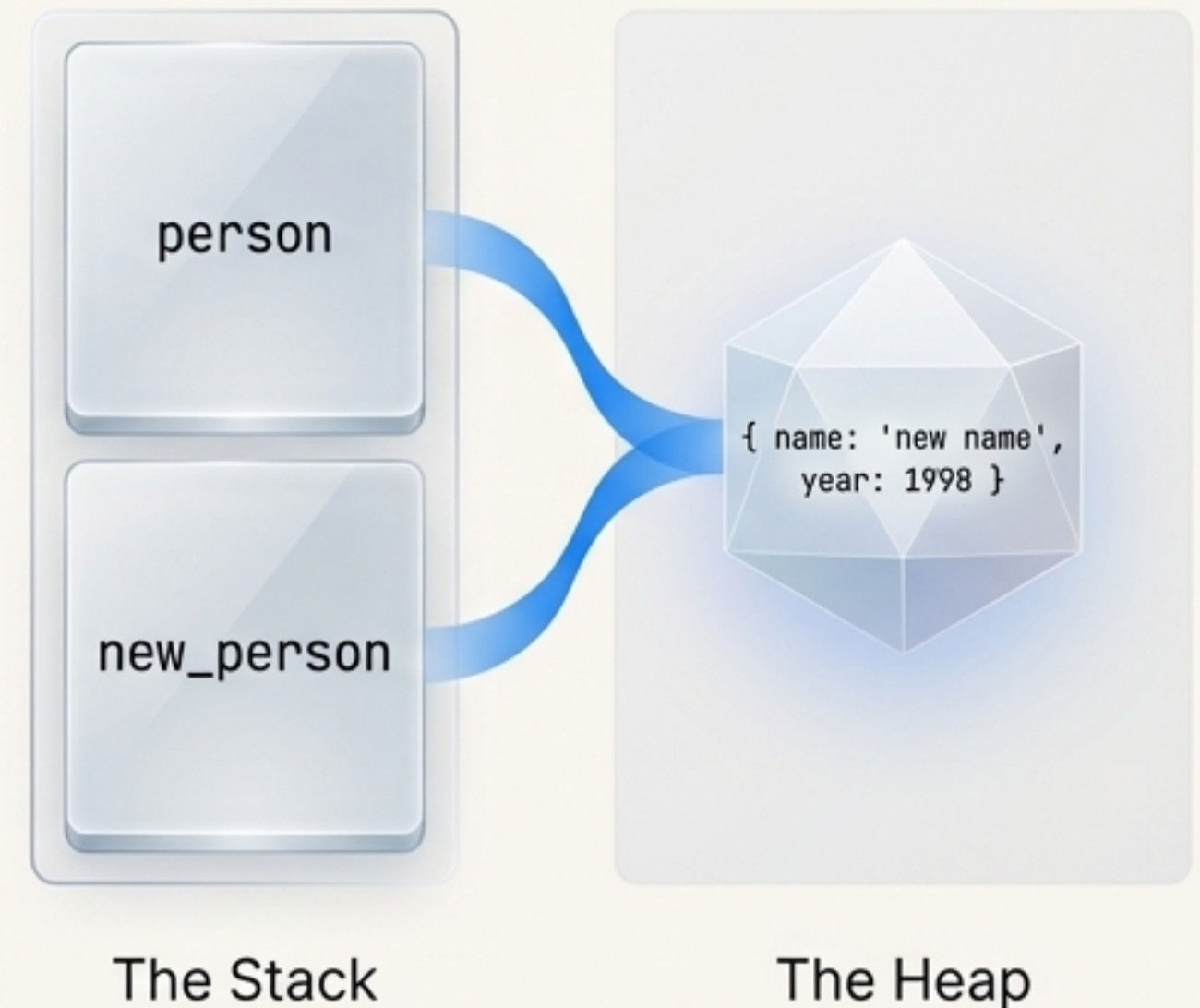
```
new_arr[1] = 500;  
arr[0] = 400;
```



This Behavior Applies to All Objects

Just like arrays, plain objects are also passed by reference. Assigning an object to a new variable creates another pointer to the same object in memory.

```
let person = { birth_year: 2000, name: "person name" };  
let new_person = person;  
  
new_person.name = "new name";  
new_person.year = 1998;  
  
console.log(person);  
// > { name: 'new name', year: 1998 }  
console.log(new_person);  
// > { name: 'new name', year: 1998 }
```



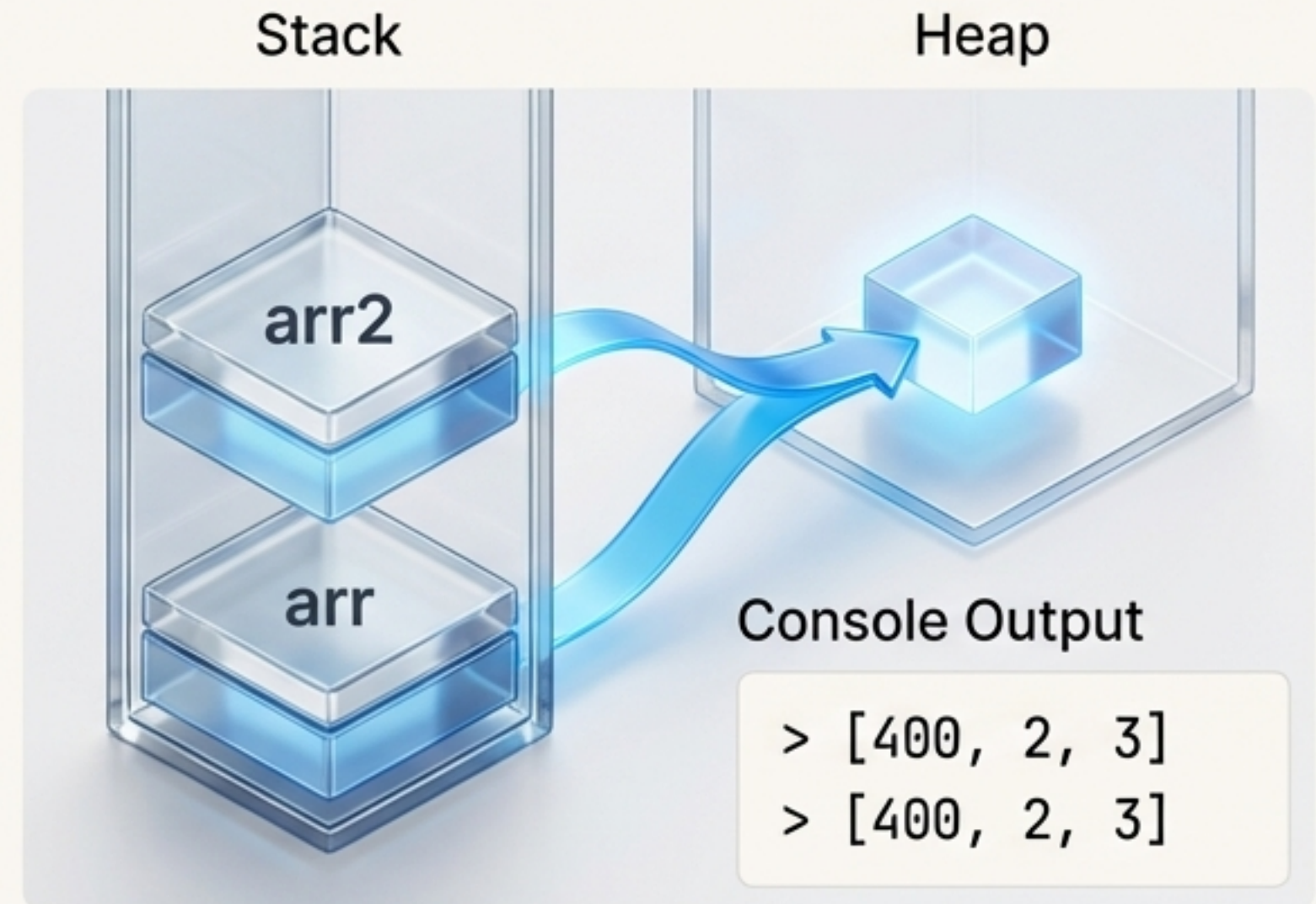
References Travel into Functions, Too

When you pass an object or array as an argument to a function, you are passing the reference. The function parameter becomes another pointer to the original object, allowing the function to mutate it directly.

```
let arr = [1, 2, 3];

function doit(arr2) {
  arr2[0] = 400;
  console.log(arr2); // Inside function
}

doit(arr);
console.log(arr);    // Outside function
```



Taking Control: How to Create a True Copy

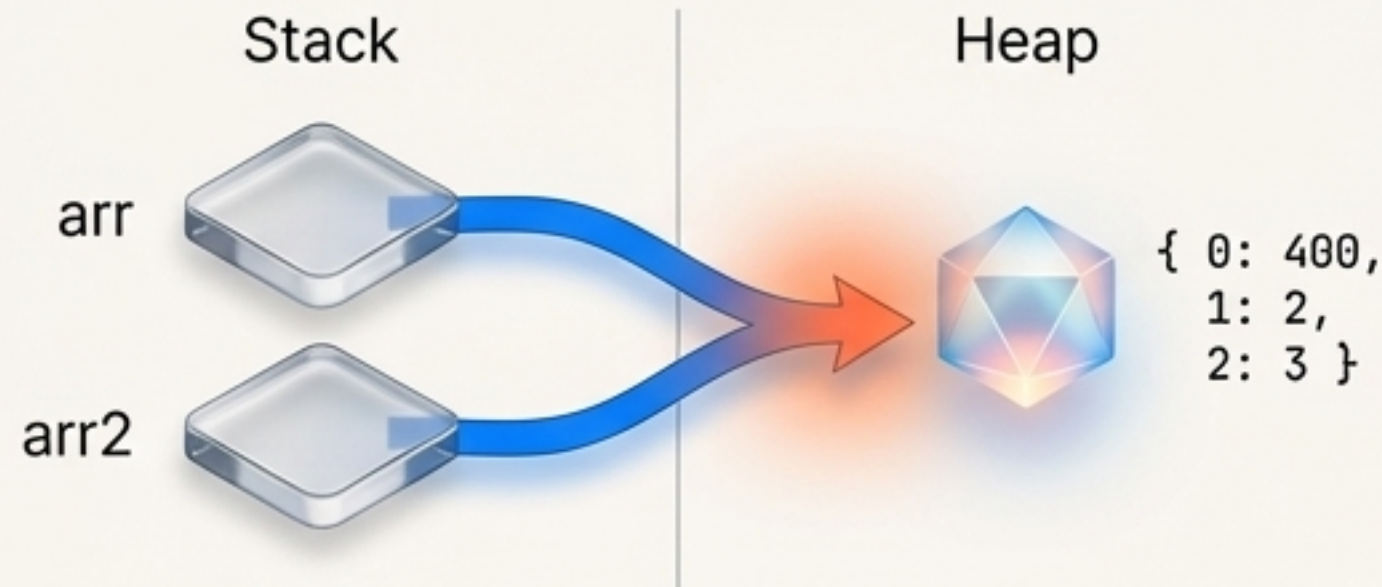
We've seen that `=` only copies references for objects. To create an independent copy and prevent unintended mutations, we must explicitly copy the object's data. JavaScript gives us tools for this. The simplest tool is the spread operator (`...`).

Pass by Reference

```
doit(arr);
```

DEBUG CONSOLE

```
> [400, 2, 3]  
> [400, 2, 3]
```

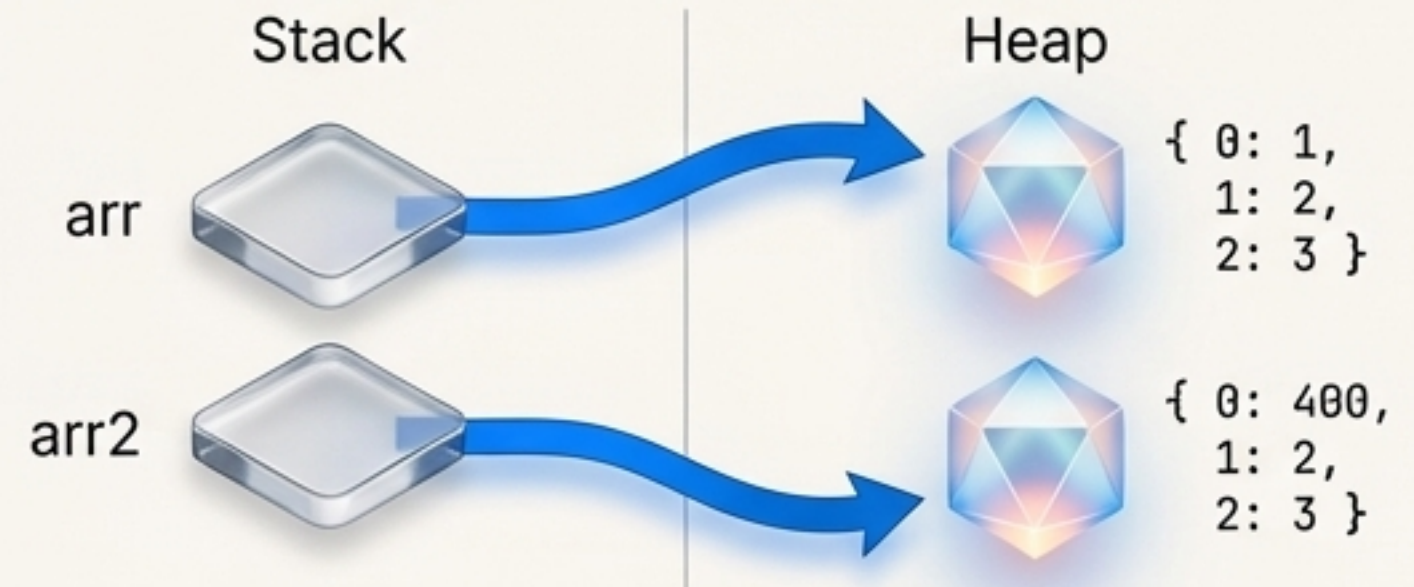


Pass by Value with Spread

```
doit([...arr]);
```

DEBUG CONSOLE

```
> [400, 2, 3] // Inside function  
> [1, 2, 3]   // Outside function
```



The Go-To Tool: Shallow Copy with the Spread Operator (...)

The spread syntax (...) unpacks an object or array's elements and places them into a new one. This creates a *new* object at the top level, breaking the reference to the original.

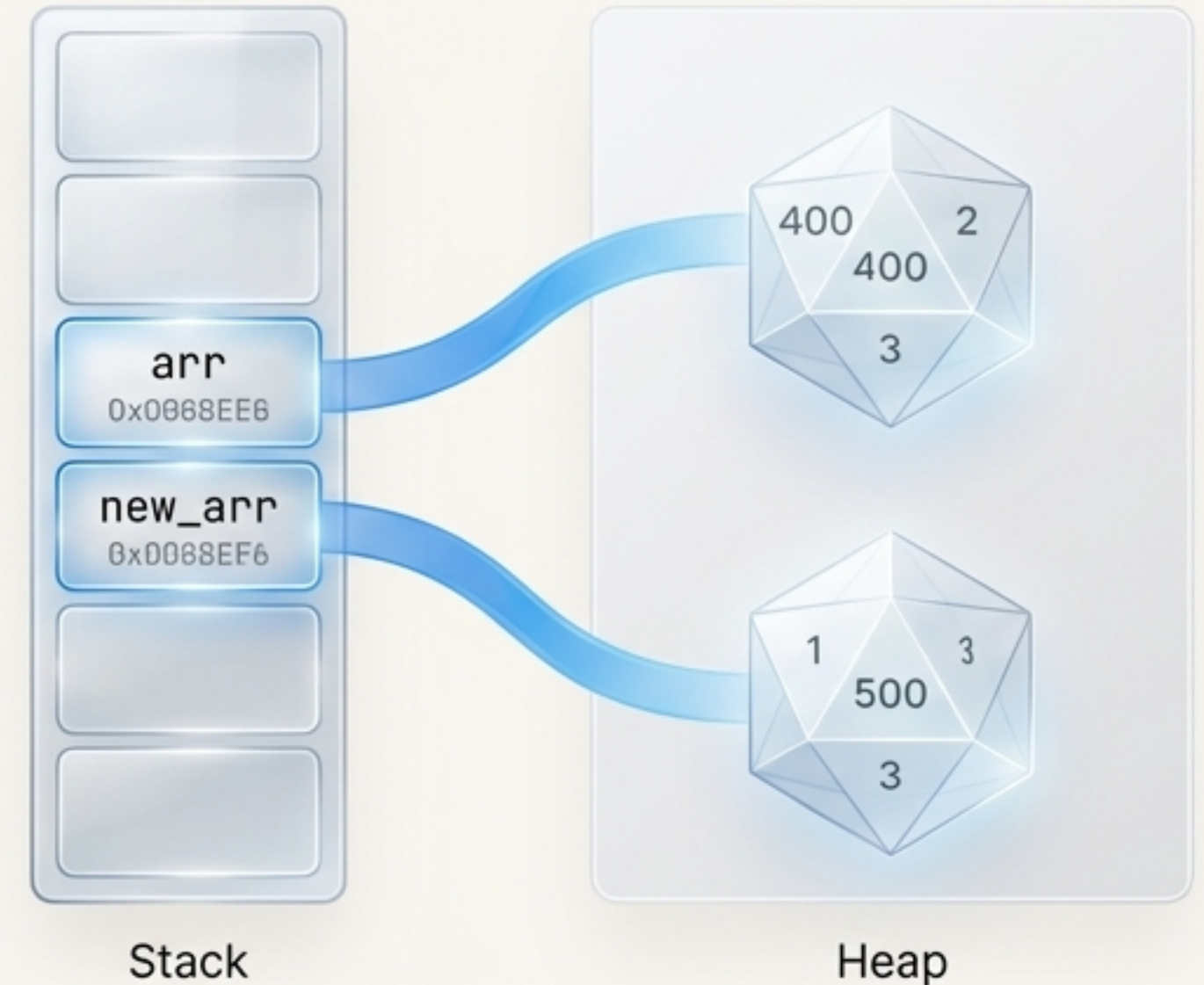
```
let arr = [1, 2, 3];  
let new_arr = [...arr]; // The only change!
```

```
new_arr[1] = 500;
```

```
arr[0] = 400;
```

```
console.log(arr);           // > [400, 2, 3]
```

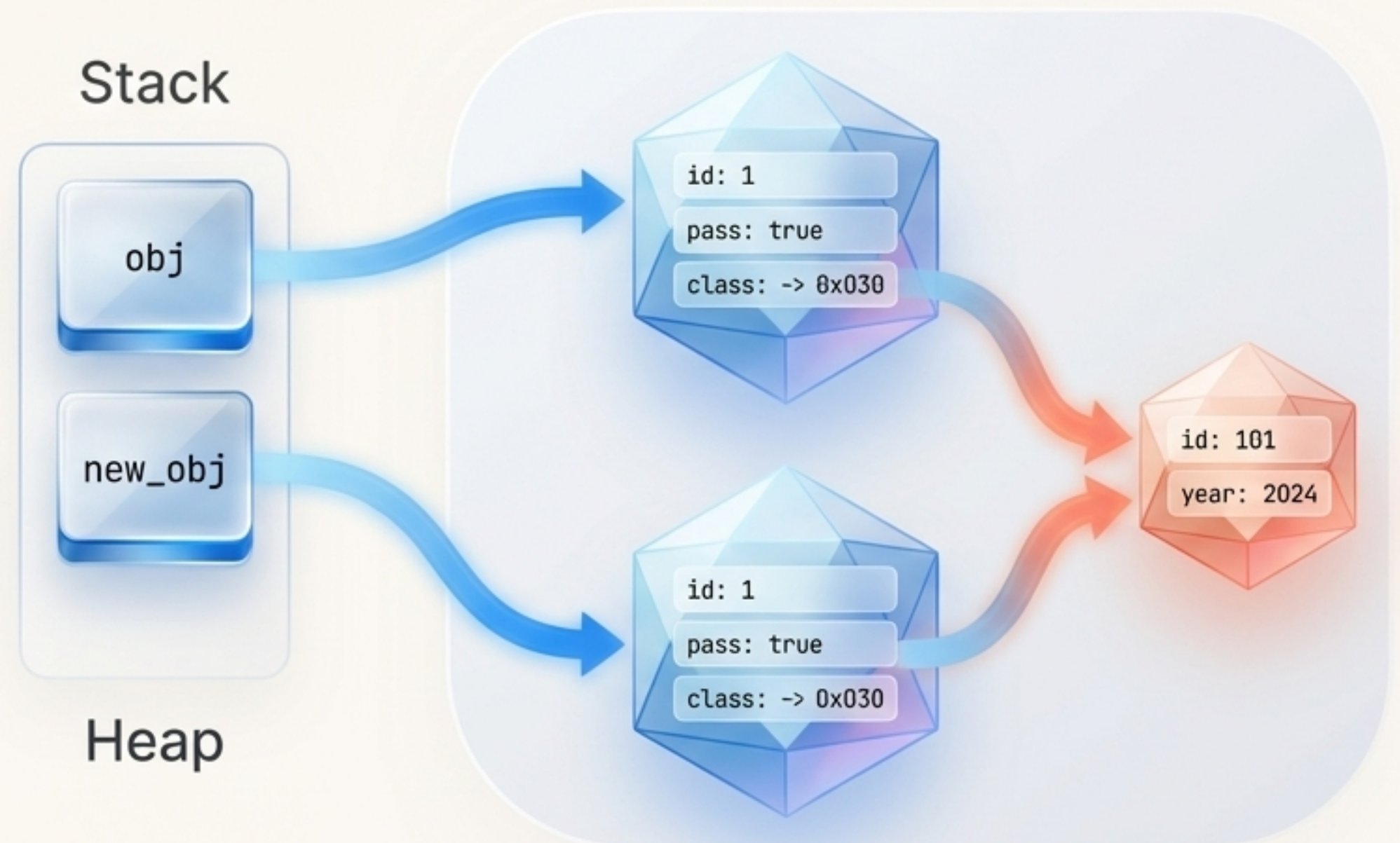
```
console.log(new_arr);       // > [1, 500, 3]
```



The Catch: A Shallow Copy is Only One Level Deep

'Shallow' means only the top-level properties are copied by value. If a property contains a nested object, only the *reference* to that nested object is copied, not the object itself.

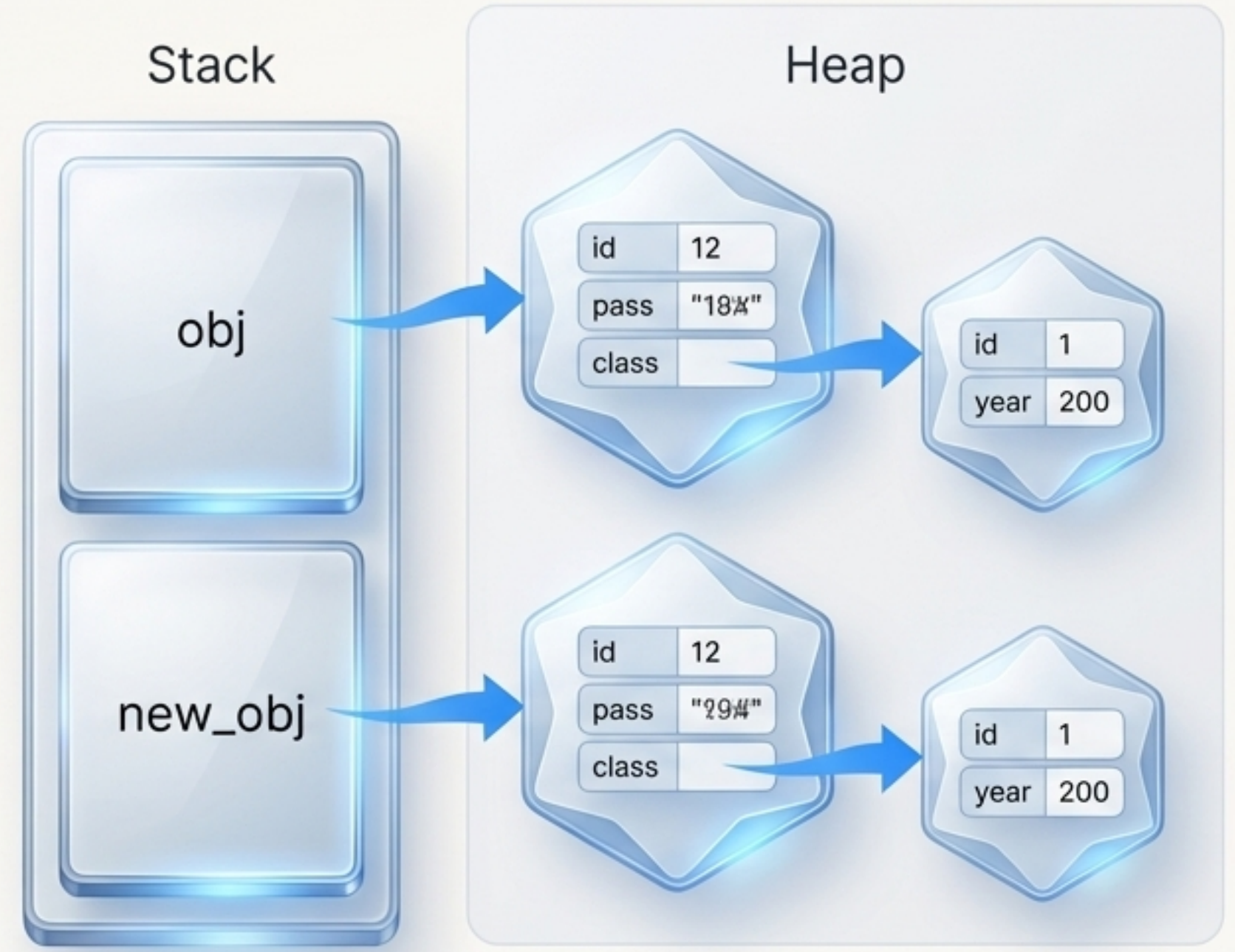
```
let obj = {  
  id: 1,  
  pass: true,  
  class: {  
    id: 101,  
    year: 2024  
  }  
};  
let new_obj = {...obj};
```



The Specialist's Tool: Deep Copy for Complete Independence

When you have nested objects and need a completely independent clone, you need a “deep copy.” This process recursively traverses the entire object, creating a new instance of every nested object and array along the way.

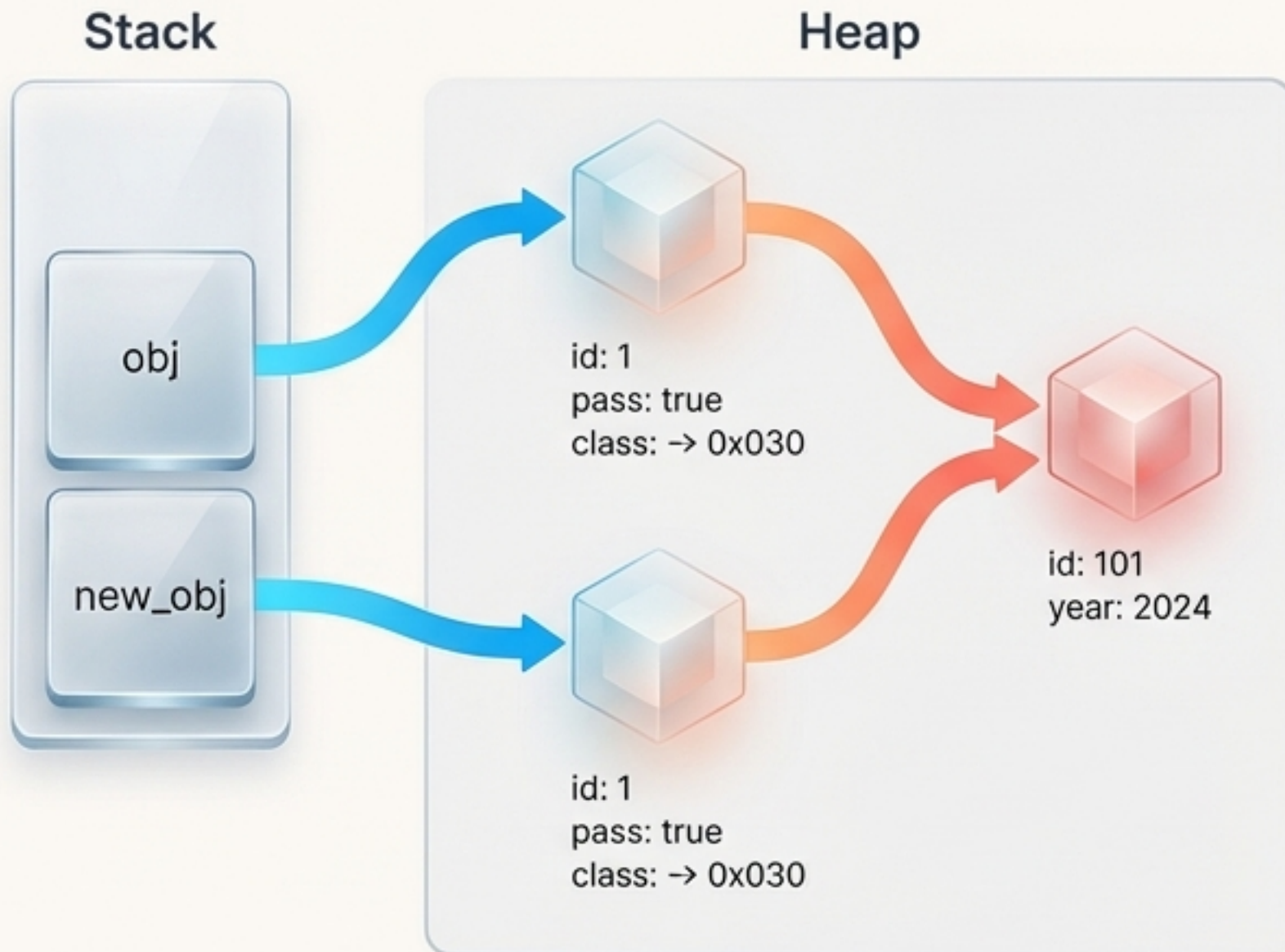
Libraries like Lodash provide `_.cloneDeep()`, or you can use `JSON.parse(JSON.stringify(obj))` for simple JSON-safe objects.



Shallow vs. Deep Copy: The Difference at a Glance

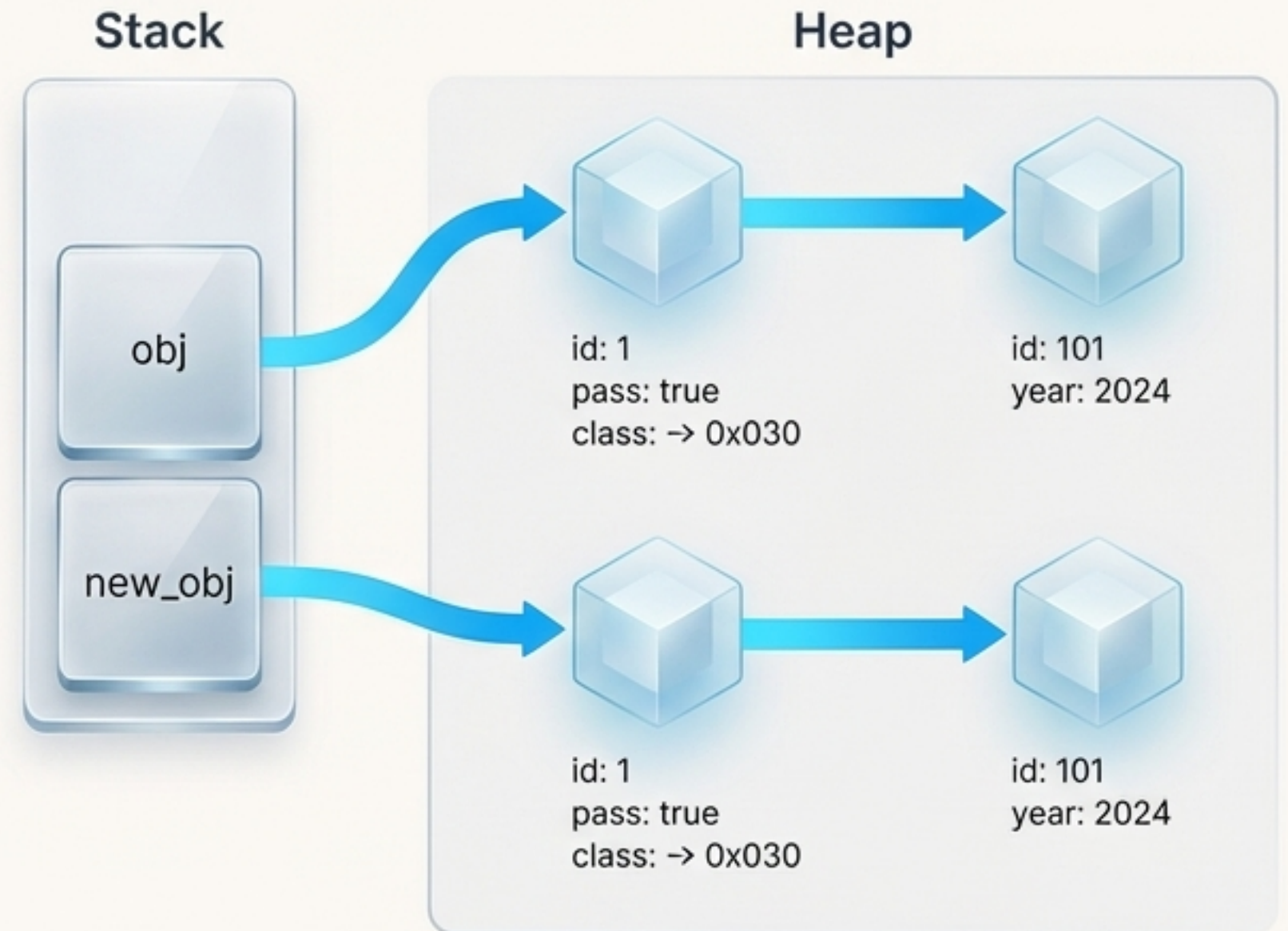
Shallow Copy

Copies the top level, shares nested levels.



Deep Copy

Copies all levels for full independence.



Mastering Memory: Your Key Takeaways



Pass by Value

- Applies to: Primitives (string, number, boolean)
- Behavior: Creates a true, independent copy. Safe from unintended changes.



Pass by Reference

- Applies to: Objects (object, array)
- Behavior: Creates a new pointer to the *same* data. Changes affect all references.



Your Toolkit

- Shallow Copy (``...``): For simple, non-nested objects. Fast and effective.
- Deep Copy: For complex, nested objects requiring full independence.